

Reviewer Pack — Minimal Local Index of GeometricALLY Enriched Group Actions ...

Entry d7491f966373 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 13:36:09 UTC

Minimal Local Index of GeometricALLY Enriched Group Actions Bounds Circuit Depth

Entry ID: d7491f966373 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 13:36:09 UTC

1. Conjecture

Field A (mathematical branch): Geometrically Enriched Group Theory **Field B** (complexity object): Boolean circuit complexity (specifically, depth)

Statement:

For every CNF φ with n variables and m clauses, the minimal local index of the geometrically enriched group action on the associated hypercube representation of φ is linearly correlated with its circuit depth $d(\varphi)$, such that $\text{mli}(G\varphi) = \Theta(d(\varphi))$, where $G\varphi$ is the geometrically enriched group action corresponding to φ .

Rationale (proposer's reasoning):

Geometrically enriched group theory provides a framework for studying discrete structures through group actions, which can offer new insights into the combinatorial properties of Boolean circuits. This conjecture suggests that the local index of such an action could serve as a measure of circuit complexity, potentially uncovering a deeper connection between geometric structure and computational difficulty.

Taxonomy category: GeometricGroupTheory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8985341a1b83fc5d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across multiple random seeds, the ratio of minimal local index to circuit depth for each CNF falls within a predefined threshold (e.g., 0.5 to 2.0).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometrically enriched group theory AND circuit depth - minimal local index IN BOOLEAN MODE AND hypercube representation AND circuit complexity - CNF formula AND geometrically enriched group action AND linear correlation with circuit depth

Top relevant hits considered: - [<http://arxiv.org/abs/hep-ph/0610012v1>] Tevatron-for-LHC Report of the QCD Working Group - [<http://arxiv.org/abs/1908.06409v1>] Schur multipliers of special p -groups of rank 2 - [<http://arxiv.org/abs/math/0608534v2>] On Automorphisms of Finite p -groups - [<http://arxiv.org/abs/2212.12756v2>] Computational Complexity of Minimal Trap Spaces in Boolean Networks - [<http://arxiv.org/abs/1208.1862v2>] A transformation rule for the index of commuting operators - [<http://arxiv.org/abs/1102.1242v2>] A refinement of Stone duality to skew Boolean algebras - [<http://arxiv.org/abs/0902.2349v1>] Defect of characters of the symmetric group - [<http://arxiv.org/abs/math/0402145v2>] Norm formulas for finite groups and induction from elementary abelian subgroups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            if random.choice([True, False]):
                clause[0] *= -1
            cnf.append(clause)
        return cnf

    def dpll(cnf):
        literals = set(abs(lit) for lit in sum(cnf, []))

        def search(model):
            unassigned = literals - {abs(lit) for lit in model}
            if not unassigned:
                return all(phi in model or -phi in model for phi in cnf)

        def search(model):
            unassigned = literals - {abs(lit) for lit in model}
            if not unassigned:
                return all(phi in model or -phi in model for phi in cnf)
```

```

        literal = next(iter(unassigned))
        pos_model = model + [literal]
        neg_model = model + [-literal]

        if search(pos_model):
            return True
        elif search(neg_model):
            return True

        return False

    return search([])

def mli(cnf):
    # Placeholder for minimal local index calculation
    # This is a dummy implementation and should be replaced with actual logic
    return random.random()

n = 10
m = 20
cnf = generate_cnf(n, m)
d_phi = dpll(cnf)
mli_phi = mli(cnf)

if d_phi == 0:
    return {
        "metric_name": "mli/d_phi",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "d_phi is zero, division by zero"
    }

ratio = mli_phi / d_phi
return {
    "metric_name": "mli/d_phi",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": 0.5 <= ratio <= 2.0,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results if result["conjecture_holds"]) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mli/d_phi ratio out of bounds' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_134aa307.py", line 87, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_134aa307.py", line 59, in run_trial
    d_phi = dpll(cnf)
             ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_134aa307.py", line 49, in dpll
    return search([])
           ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_134aa307.py", line 42, in search
    if search(pos_model):
        ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_134aa307.py", line 42, in search
    if search(pos_model):
        ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_134aa307.py", line 42, in search
    if search(pos_model):
        ^^^^^^^^^^^^^^^
  [Previous line repeated 7 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_134aa307.py", line 36, in search
    return all(phi in model or -phi in model for phi in cnf)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_134aa307.py", line 36, in <genexpr>
    return all(phi in model or -phi in model for phi in cnf)
           ^^^^^
TypeError: bad operand type for unary -: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test with appropriate error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	28112
2	propose	ollama_remote	glm4:latest	0	0	26496
3	preregistration	ollama_remote	glm4:latest	0	0	15616
4	novelty	ollama_remote	glm4:latest	0	0	12242
5	novelty	ollama_remote	glm4:latest	0	0	11251
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12858
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10445
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19271
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8871
10	judge	ollama_remote	glm4:latest	0	0	8805

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 153966 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d7491f966373.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d7491f966373.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d7491f966373.tar.gz` (if generated)